

Compute-Optionen: Cluster-Typen, Kosten und Workload-Auswahl

Section 1 · Databricks Intelligence Plattform · Quelle: Databricks-Dokumentation (Compute & Pricing)

Databricks bietet mehrere Compute-Typen mit unterschiedlichen Eigenschaften, Kostenmodellen und Einsatzzwecken an. Die richtige Wahl zu treffen ist eine eigenständige Prüfungskompetenz: Es geht nicht nur darum, dass ein Workload überhaupt läuft, sondern dass er auf der wirtschaftlich und technisch passenden Infrastruktur läuft.

1. All-Purpose Compute

All-Purpose-Cluster sind für interaktive Entwicklung gedacht: mehrere Personen können sich denselben Cluster teilen, um Notebooks zu schreiben, zu explorieren und Ad-hoc-Abfragen auszuführen. Sie bleiben so lange laufen, bis sie manuell gestoppt oder durch eine konfigurierte Inaktivitäts-Timeout-Regel automatisch heruntergefahren werden. Der Kompromiss: Der DBU-Satz (Databricks Unit, die Verrechnungseinheit für Compute) ist hier am höchsten, da neben reiner Rechenleistung auch interaktive Funktionen wie Autovervollständigung, Notebook-Anhänge und gemeinsame Nutzung mitfinanziert werden.

2. Job Compute

Job-Cluster werden ausschließlich für die Ausführung eines geplanten oder ausgelösten Lakeflow Jobs erzeugt und nach Abschluss automatisch wieder terminiert – es gibt keine Leerlaufzeit, für die bezahlt werden müsste. Der DBU-Satz liegt deutlich unter dem von All-Purpose-Clustern, da keine interaktiven Zusatzfunktionen benötigt werden. Job-Cluster sind damit die wirtschaftlich richtige Wahl für produktive, wiederkehrende ETL-Workloads – nicht für Entwicklung und Exploration.

3. SQL-Warehouses

SQL-Warehouses sind speziell für SQL-Analytics- und BI-Workloads optimiert (Dashboards, Power BI, Tableau, Ad-hoc-SQL-Abfragen vieler gleichzeitiger Nutzer). Sie unterstützen automatische

Skalierung über mehrere Cluster hinweg (Multi-Cluster-Load-Balancing) für hohe Nebenläufigkeit und sind in einer klassischen (provisionierten) sowie einer serverlosen Variante verfügbar.

4. Serverless Compute

Serverless Compute ist inzwischen für alle Compute-Typen verfügbar – Notebooks, Jobs und SQL-Warehouses. Databricks verwaltet dabei die zugrunde liegende Infrastruktur (Instanztypen, Skalierung, Patches) vollständig selbst; Nutzerinnen und Nutzer zahlen nur für tatsächlich verbrauchte Rechenzeit. Der größte praktische Vorteil ist die Startzeit: Serverless-SQL-Warehouses stehen typischerweise innerhalb weniger Sekunden bereit, während klassische Cluster mehrere Minuten zum Hochfahren benötigen.

```
# Compute-Konfiguration ist deklarativ, z. B. innerhalb eines Jobs (Auszug aus
resources/job.yml)
# job_clusters:
#   - job_cluster_key: etl_cluster
#     new_cluster:
#       spark_version: "15.4.x-scala2.12"
#       node_type_id: "Standard_DS3_v2"
#       autoscale:
#         min_workers: 2
#         max_workers: 8
```

5. Entscheidungskriterien für die Prüfung

Anforderung	Passender Compute-Typ
Mehrere Personen entwickeln/explorieren interaktiv im selben Notebook	All-Purpose Compute
Geplanter, wiederkehrender ETL-Job, kein interaktiver Zugriff nötig	Job Compute (klassisch oder serverless)
Viele gleichzeitige BI-/Dashboard-Nutzer mit variabler Last	SQL-Warehouse mit Autoscaling/Multi-Cluster
Schnelle Startzeit, minimaler Verwaltungsaufwand, variable Auslastung	Serverless Compute (beliebiger Typ)
Kosten für Leerlaufzeit strikt vermeiden	Job Compute oder Serverless (kein Dauerbetrieb)

Ergänzt aus der offiziellen Databricks-Dokumentation: Der DBU-Satz unterscheidet sich zwischen All-Purpose-, Job- und SQL-Warehouse-Compute teils um mehr als das Dreifache für denselben zugrunde liegenden Instanztyp – ein häufiger Kostenfehler ist, aus Gewohnheit alle Workloads auf All-Purpose-Clustern laufen zu lassen, obwohl produktive, nicht-interaktive Jobs auf Job-Compute deutlich günstiger wären. Serverless-SQL-Warehouses sind laut Databricks innerhalb weniger Sekunden startklar, was insbesondere bei sporadischer BI-Nutzung Kosten spart, da keine dauerhaft laufende Infrastruktur vorgehalten werden muss.

Quelle: [Databricks Pricing & DBU-Modell – Databricks](#)